Text as Data

Class 5: Supervised modelling

Petro Tolochko

CEU

Fall 2026

Machine Learning

Machine Learning

- Supervised
  - An outcome variable is defined
  - Focus on prediction

Machine Learning

- Supervised
  - An outcome variable is defined
  - Focus on prediction
- Usupervised
  - No outcome variable defined
  - Focus on pattern recognition

Machine Learning

- **Supervised**
  - An outcome variable is defined
  - Focus on prediction
- Unsupervised
  - No outcome variable defined
  - Focus on pattern recognition

Supervised

- Objective:

Supervised

- Objective:
  - Classification of documents into preexisting categories

Supervised

- Objective:
  - Classification of documents into preexisting categories
  - Text regression (scaling)

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods

Supervised Machine Learning

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Learns patterns from labeled examples

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Meaning inferred from word presence

- Learns patterns from labeled examples
- Uses features (e.g., words, embeddings)

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Meaning inferred from word presence
- Transparent and interpretable

- Learns patterns from labeled examples
- Uses features (e.g., words, embeddings)
- Often less interpretable (black box)

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Meaning inferred from word presence
- Transparent and interpretable
- No training data required

- Learns patterns from labeled examples
- Uses features (e.g., words, embeddings)
- Often less interpretable (black box)
- Requires annotated training data

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Meaning inferred from word presence
- Transparent and interpretable
- No training data required
- Limited adaptability (depends on dictionary)

- Learns patterns from labeled examples
- Uses features (e.g., words, embeddings)
- Often less interpretable (black box)
- Requires annotated training data
- Flexible and domain-adaptive

Dictionary vs. Supervised Machine Learning

Dictionary-based Methods $\Leftrightarrow$ **Supervised Machine Learning**

- Use predefined word lists (lexicons)
- Meaning inferred from word presence
- Transparent and interpretable
- No training data required
- Limited adaptability (depends on dictionary)

- Learns patterns from labeled examples
- Uses features (e.g., words, embeddings)
- Often less interpretable (black box)
- Requires annotated training data
- Flexible and domain-adaptive

$\rightarrow$ *Dictionary methods encode human theory; supervised ML discovers patterns from data.*

Supervised ML Pipeline

- Create a labeled dataset
- Apply a function that maps features $\rightarrow$ outcome (ML step)
- Assess performance

Classifying Documents with Supervised Learning

- **Key considerations:**

Classifying Documents with Supervised Learning

- **Key considerations:**
  - **Feature representation:** how to turn text into numbers (e.g., Bag-of-Words, TF–IDF, or embeddings)

Classifying Documents with Supervised Learning

- **Key considerations:**
  - **Feature representation:** how to turn text into numbers (e.g., Bag-of-Words, TF–IDF, or embeddings)
  - **Feature selection:** remove noisy or irrelevant features (e.g., stop words, rare terms)

Classifying Documents with Supervised Learning

- **Key considerations:**
  - **Feature representation:** how to turn text into numbers (e.g., Bag-of-Words, TF–IDF, or embeddings)
  - **Feature selection:** remove noisy or irrelevant features (e.g., stop words, rare terms)
  - **Classifier selection:** choose the learning algorithm (e.g., Naive Bayes, SVM, k-NN, Logistic Regression, or ensembles)

Classifying Documents with Supervised Learning

- **Key considerations:**
  - **Feature representation:** how to turn text into numbers (e.g., Bag-of-Words, TF–IDF, or embeddings)
  - **Feature selection:** remove noisy or irrelevant features (e.g., stop words, rare terms)
  - **Classifier selection:** choose the learning algorithm (e.g., Naive Bayes, SVM, k-NN, Logistic Regression, or ensembles)

- **Goal:** learn a function $f(X) \rightarrow Y$ that generalizes to unseen documents.

Labeled Dataset

- How:
  - Human coders annotate parts of the corpus
  - Found data (e.g., self-reported profession in users' profile)
- Considerations:
  - Sampling should be representative for the corpus (e.g., Random, Stratified sample e.g., across time and source)
  - Quality of human coding matters (Assess the intercoder reliability)
  - Number of documents

Labeled Dataset

- Number of documents
  - the higher the number of categories and the lower the reliability of the coders, the higher the number of documents (Barberá et al., 2021)
- increase the sizes of manually coded validation dataset as large as possible (e.g., more than 1% of all data to be examined), assuming acceptable reliability (equal to or higher than .7) (Song et al., 2021)

Splitting the Data

- Split labeled data in training data and test data (validation data)

Splitting the Data

- Split labeled data in training data and test data (validation data)
- Training Data
  - The subset that is used to learn the model parameters

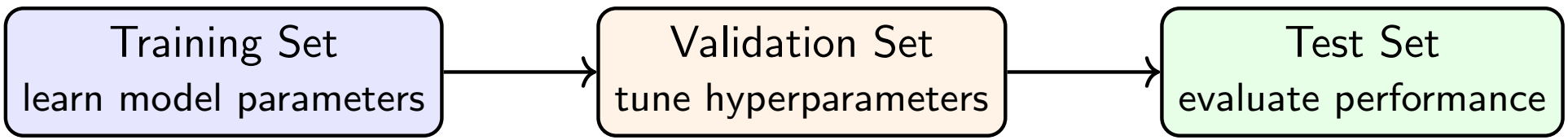
Splitting the Data

- Split labeled data in training data and test data (validation data)
- Training Data
  - The subset that is used to learn the model parameters
- Validation Data
  - The subset used to tune validate the parameters

Splitting the Data

- Split labeled data in training data and test data (validation data)
- Training Data
  - The subset that is used to learn the model parameters
- Validation Data
  - The subset used to tune validate the parameters
- Test Data
  - The subset that is used to evaluate the final performance
  - Not used for learning

Train–Validation–Test Split



Used during model development → Final evaluation only

The validation set guides learning; the test set measures generalization.

Cross-Validation

- Used to estimate model performance when data is limited.

Cross-Validation

- Used to estimate model performance when data is limited.
- Split data into k folds (subsets).

Cross-Validation

- Used to estimate model performance when data is limited.
- Split data into k folds (subsets).
- Train the model on $k-1$ folds and validate on the remaining one.

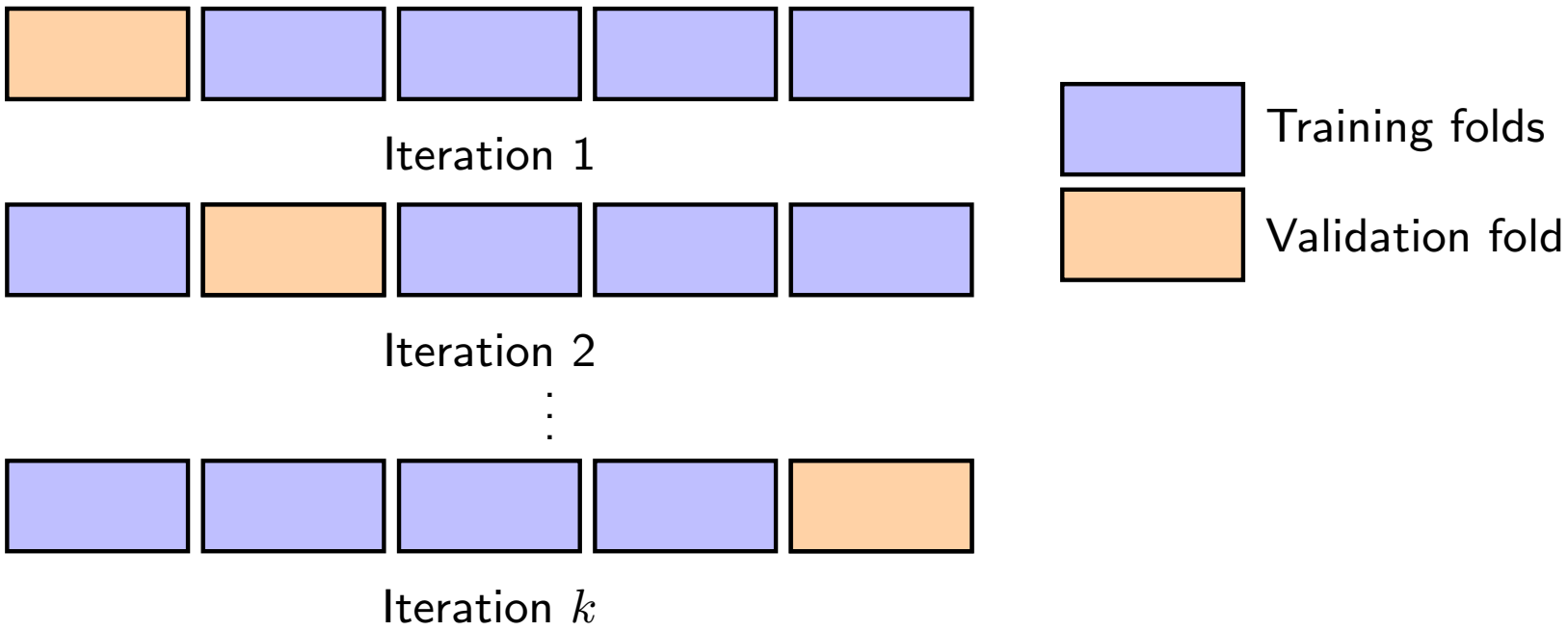
Cross-Validation

- Used to estimate model performance when data is limited.
- Split data into k folds (subsets).
- Train the model on $k-1$ folds and validate on the remaining one.
- Repeat k times, each time with a different validation fold.

Cross-Validation

- Used to estimate model performance when data is limited.
- Split data into k folds (subsets).
- Train the model on $k-1$ folds and validate on the remaining one.
- Repeat k times, each time with a different validation fold.
- Average performance across folds $\Rightarrow$ robust estimate of generalization.

Cross-Validation



Each fold is used once for validation and $k-1$ times for training.

Preprocessing leakage

The training split must determine which features the model can learn from.

- Fit vocabulary, document-frequency filters, and IDF values on training text.
- Transform validation and test texts with that fitted representation.
- In cross-validation, refit those steps inside each training fold.

Choosing words after inspecting test labels or repeatedly tuning against test scores also lets the test set guide the model.

A pipeline helps keep representation fitting and classifier fitting together.

Document Classification

- Classifier *learns* the mapping between the features and the labels in the training set

Document Classification

- Classifier *learns* the mapping between the features and the labels in the training set
- Define a model $Y = f(X)$

Document Classification

- Classifier *learns* the mapping between the features and the labels in the training set
- Define a model $Y = f(X)$
- Apply the model to *learn* which features in X (extracted from raw text) matter to recover Y (i.e., labels from the training data)

Machine Learning 101

Machine Learning 101

- **Model:**

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$
- **Objective function:**

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$
- **Objective function:**
  - $\mathcal{L}(\theta; \mathcal{D})$
    - $\mathcal{L}$ – generic loss function
    - $\mathcal{D}$ – data
    - θ – parameter vector

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$
- **Objective function:**
  - $\mathcal{L}(\theta; \mathcal{D})$
    - $\mathcal{L}$ – generic loss function
    - $\mathcal{D}$ – data
    - θ – parameter vector
    - Example: $MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$
- **Objective function:**
  - $\mathcal{L}(\theta; \mathcal{D})$
    - $\mathcal{L}$ – generic loss function
    - $\mathcal{D}$ – data
    - θ – parameter vector
    - Example: $MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$
- **Optimization step:**
  - $\hat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D})$

Machine Learning 101

- **Model:**
  - $Y = f(X; \theta)$
- **Objective function:**
  - $\mathcal{L}(\theta; \mathcal{D})$
    - $\mathcal{L}$ – generic loss function
    - $\mathcal{D}$ – data
    - θ – parameter vector
    - Example: $MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$
- **Optimization step:**
  - $\hat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D})$
  - For example: $\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} (y_i - f_\theta(x_i))^2$

Machine Learning 101

- **Model (Machine):**
  - $Y = f(X; \theta)$
- **Objective function (Learning):**
  - $\mathcal{L}(\theta; \mathcal{D})$
    - $\mathcal{L}$ – generic loss function
    - $\mathcal{D}$ – data
    - θ – parameter vector
    - Example: $MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$
- **Optimization step (Learning):**
  - $\hat{\theta} = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D})$
  - For example: $\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} (y_i - f_\theta(x_i))^2$

Questions?

Naive Bayes

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

- $P(A)$ — prior probability of hypothesis A

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

- $P(A)$ — prior probability of hypothesis A
- $P(B \mid A)$ — likelihood of observing B if A is true

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

- $P(A)$ — prior probability of hypothesis A
- $P(B \mid A)$ — likelihood of observing B if A is true
- $P(B)$ — marginal probability of observing B

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

- $P(A)$ — prior probability of hypothesis A
- $P(B \mid A)$ — likelihood of observing B if A is true
- $P(B)$ — marginal probability of observing B
- $P(A \mid B)$ — posterior probability after seeing B

Bayes' Rule

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

- $P(A)$ — prior probability of hypothesis A
- $P(B \mid A)$ — likelihood of observing B if A is true
- $P(B)$ — marginal probability of observing B
- $P(A \mid B)$ — posterior probability after seeing B

Bayesian idea: update prior beliefs with new evidence.

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

- C: class label (e.g., positive/negative sentiment)

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

- C: class label (e.g., positive/negative sentiment)
- X: document represented as words or features

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

- C: class label (e.g., positive/negative sentiment)
- X: document represented as words or features
- Naive assumption: features are conditionally independent given the class

$$P(X \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

- C: class label (e.g., positive/negative sentiment)
- X: document represented as words or features
- Naive assumption: features are conditionally independent given the class

$$P(X \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

- In text classification: estimate $P(x_i \mid C)$ from word frequencies.

From Bayes' Rule to the Naive Bayes Classifier

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)}$$

$$\Rightarrow \quad \hat{C} = \arg\max_{C} P(X \mid C)\, P(C)$$

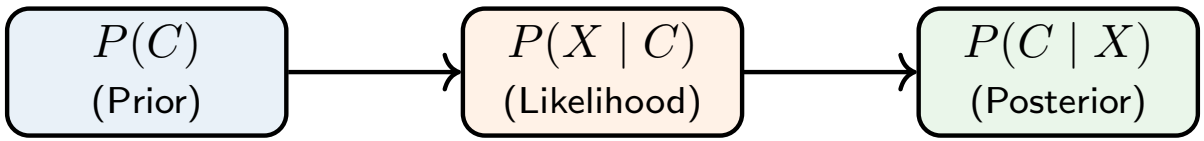
- C: class label (e.g., positive/negative sentiment)
- X: document represented as words or features
- Naive assumption: features are conditionally independent given the class

$$P(X \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

- In text classification: estimate $P(x_i \mid C)$ from word frequencies.

$\rightarrow$ *The classifier picks the class with the highest posterior probability.*

How Priors and Likelihoods Combine



Posterior $=$ *Prior* $\times$ *Likelihood (then normalized).*

Naive Bayes picks the class with the highest posterior.

Naive Bayes and the Multinomial Model of Language

- The **Naive Bayes classifier** assumes conditional independence of features:

$$P(X \mid C) = \prod_{i=1}^{V} P(x_i \mid C)$$

Naive Bayes and the Multinomial Model of Language

- The **Naive Bayes classifier** assumes conditional independence of features:

$$P(X \mid C) = \prod_{i=1}^{V} P(x_i \mid C)$$

- For text, we use a **multinomial model**:

$$P(X \mid C) = \frac{N!}{x_1! \ldots x_V!} \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

Naive Bayes and the Multinomial Model of Language

- The **Naive Bayes classifier** assumes conditional independence of features:

$$P(X \mid C) = \prod_{i=1}^{V} P(x_i \mid C)$$

- For text, we use a **multinomial model**:

$$P(X \mid C) = \frac{N!}{x_1! \ldots x_V!} \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

- Each class defines its own word distribution $P(w_i \mid C)$.

Naive Bayes and the Multinomial Model of Language

- The **Naive Bayes classifier** assumes conditional independence of features:

$$P(X \mid C) = \prod_{i=1}^{V} P(x_i \mid C)$$

- For text, we use a **multinomial model**:

$$P(X \mid C) = \frac{N!}{x_1! \ldots x_V!} \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

- Each class defines its own word distribution $P(w_i \mid C)$.
- Classification rule:

$$\hat{C} = \arg\max_C P(C) \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

Naive Bayes and the Multinomial Model of Language

- The **Naive Bayes classifier** assumes conditional independence of features:

$$P(X \mid C) = \prod_{i=1}^{V} P(x_i \mid C)$$

- For text, we use a **multinomial model**:

$$P(X \mid C) = \frac{N!}{x_1! \dots x_V!} \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

- Each class defines its own word distribution $P(w_i \mid C)$.
- Classification rule:

$$\hat{C} = \arg\max_C P(C) \prod_{i=1}^{V} P(w_i \mid C)^{x_i}$$

- Interpretation: each class is a simple *language model* — choose the one most likely to have generated the document.

Questions?

A worked Naive Bayes example

Training class	great	dull	plot	Total
Positive	8	2	5	15
Negative	1	7	7	15

This invented example has a vocabulary of three words. Add-one smoothing gives

$$\theta_{cw} = \frac{n_{cw} + 1}{15 + 3}.$$

$$\boldsymbol{\theta}_+ = (9/18,\ 3/18,\ 6/18), \qquad \boldsymbol{\theta}_- = (2/18,\ 8/18,\ 8/18).$$

Each row sums to one. Assume equal class priors, $\pi_+ = \pi_- = 0.5$.

Predicting *great plot*

The count vector is $\mathbf{x} = (1, 0, 1)$. Class scores are

$$S_+ = 0.5(9/18)(6/18) = 1/12,$$
$$S_- = 0.5(2/18)(8/18) = 2/81.$$

The multinomial coefficient is identical for both classes and cancels when we normalize.

$$P(+ \mid \mathbf{x}) = \frac{S_+}{S_+ + S_-} = \frac{27}{35} \approx 0.771.$$

The model predicts the positive class. This calculation follows the word model and priors we specified.

Naive Bayes as learned word weights

For two classes, subtract their log scores:

$$\log \frac{P(+ \mid \mathbf{x})}{P(- \mid \mathbf{x})} = \underbrace{\log \frac{\pi_+}{\pi_-}}_{\text{intercept}} + \sum_w x_w \underbrace{\log \frac{\theta_{+w}}{\theta_{-w}}}_{\text{word weight}} .$$

A word contributes positive evidence when it is more probable in the positive class.

This resembles a weighted dictionary score. The weights are estimated from labelled examples rather than assigned by hand.

The formula connects multinomial word probabilities to the log odds introduced earlier in the course.

Logistic regression for text

Logistic regression also computes a weighted document score:

$$z_i = b + \mathbf{x}_i^{\mathsf{T}} \boldsymbol{\beta}.$$

It maps that score to a positive-class probability:

$$p_i = P(y_i = 1 \mid \mathbf{x}_i) = \frac{1}{1 + \exp(-z_i)}.$$

$$z_i = -2 \Rightarrow p_i \approx 0.119, \quad z_i = 0 \Rightarrow p_i = 0.5, \quad z_i = 2 \Rightarrow p_i \approx 0.881.$$

A positive coefficient supplies evidence toward the positive label. For a particular text, its contribution is the feature value multiplied by that coefficient.

Learning weights through log loss

For a training label $y_i \in \{0, 1\}$ and prediction p_i:

$$\ell(y_i, p_i) = -\left[y_i \log p_i + (1 - y_i) \log(1 - p_i)\right].$$

Predicted probability of positive, if $y = 1$	Log loss
0.1	2.303
0.5	0.693
0.9	0.105

A confident incorrect prediction receives a large loss. Logistic regression learns weights that reduce this loss over labelled examples.

Regularization

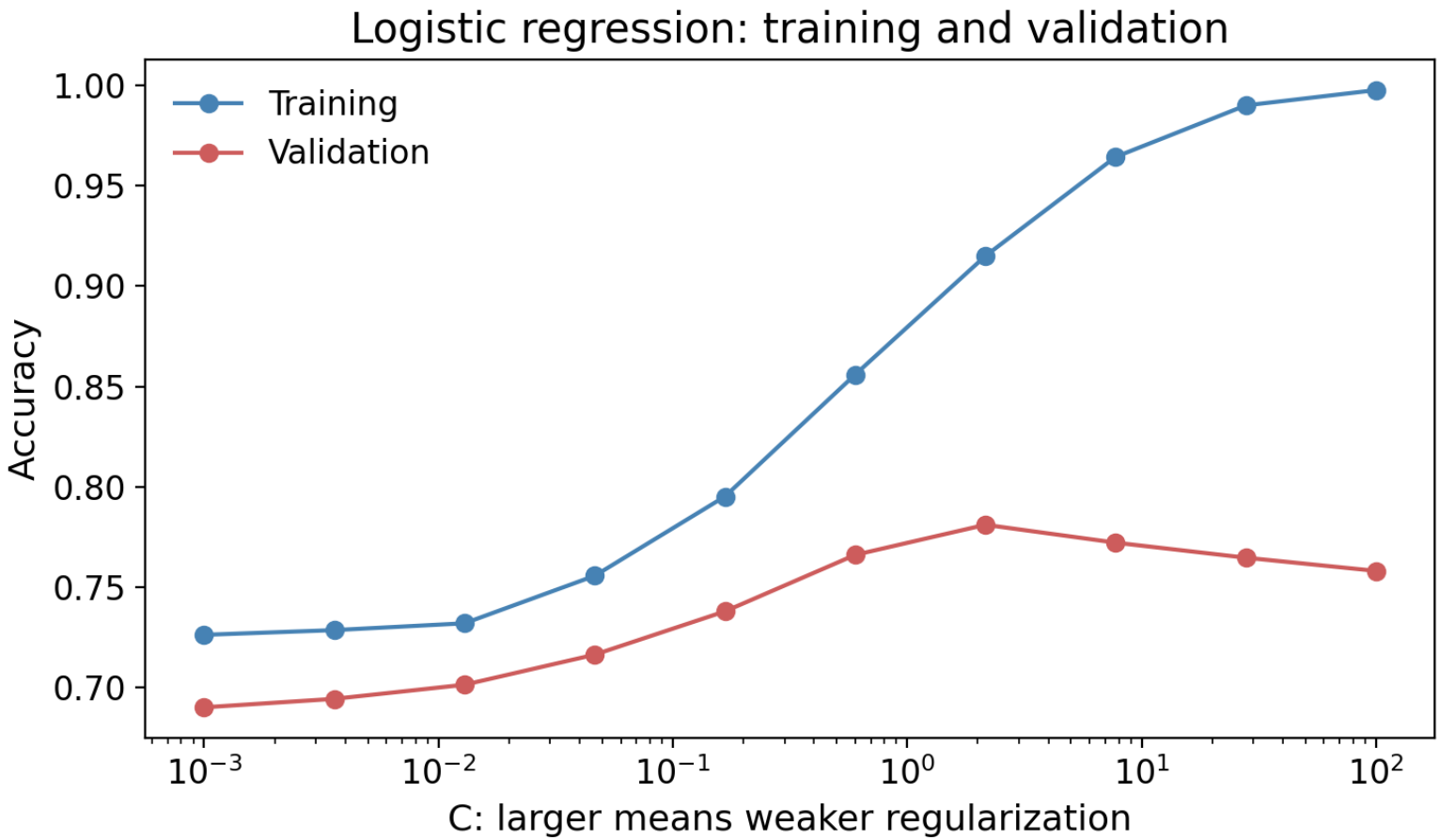
Reduce prediction error while penalizing large feature weights:

$$\mathcal{L}_{\text{reg}} = \frac{1}{N}\sum_{i=1}^{N} \ell(y_i, p_i) + \frac{\lambda}{2}\|\boldsymbol{\beta}\|_2^2.$$

- A stronger penalty encourages smaller coefficients.
- This limits reliance on weakly supported words or phrases.
- Too much regularization can suppress useful differences.

The notebook uses C, an inverse regularization setting: smaller C means a stronger penalty. Choose it using validation data.

Training fit and overfitting



Weakening the penalty can improve training accuracy while validation accuracy levels off or falls. The training score alone cannot select the model.

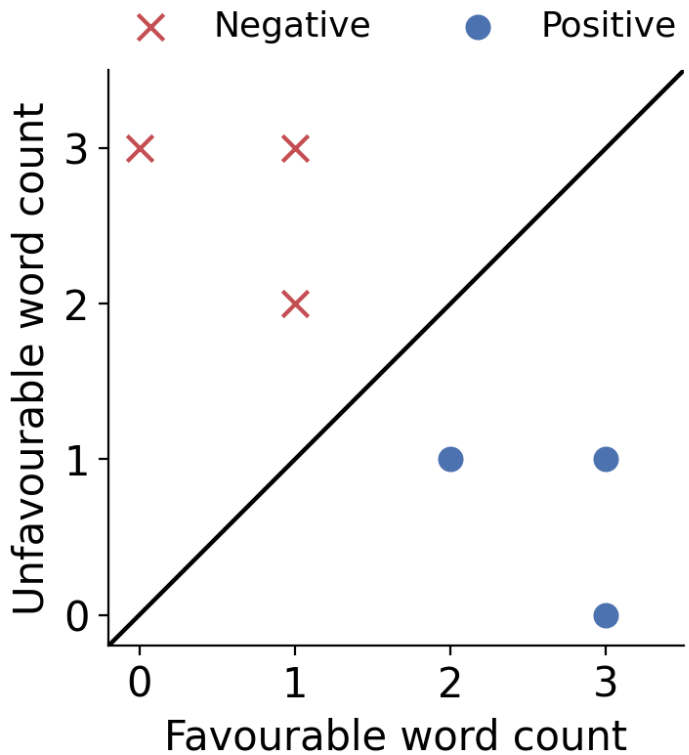
Rotten Tomatoes logistic-regression example in the notebook. TF-IDF is fitted on training texts only.

Linear support vector machines

A linear SVM learns a boundary that separates the document vectors.

It favours a wide gap between the boundary and nearby training examples.

The plot shows two illustrative word-count features. Real text vectors can have thousands of features.



A linear classification rule

Each word feature receives a weight, and the model computes

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b.$$

The decision boundary is where $z = 0$. A binary prediction is

$$\hat{y} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$

Positive scores place the document on the positive side of the boundary. Negative scores place it on the negative side.

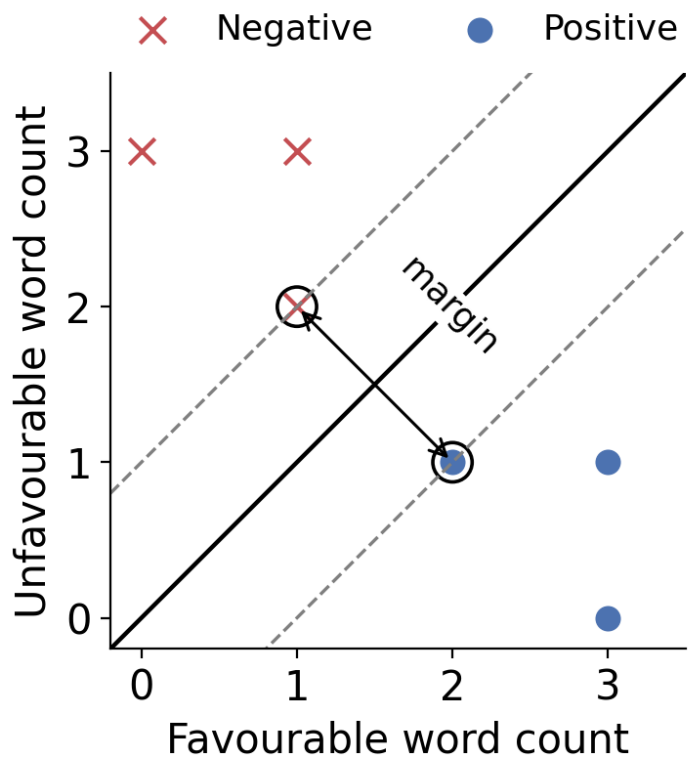
For $\mathbf{w} = (1, -1)$ and $b = 0$, more favourable than unfavourable terms produce a positive score.

Margins and support vectors

The margin is the gap between the closest examples of the two classes around a separating boundary.

For perfectly separable data, the closest points are the support vectors. They determine the maximum-margin separator.

The circled points support the margin in this example.



SVMs with overlapping classes

Real texts do not always separate perfectly. A soft-margin SVM balances a wide margin against violations of that margin.

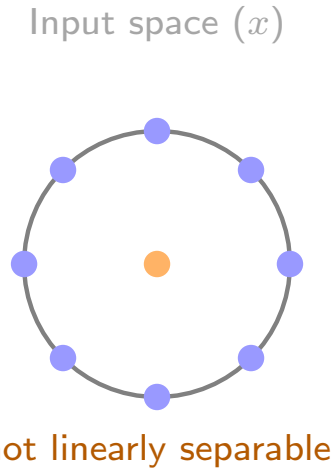
- A larger C penalizes margin violations more strongly.
- A smaller C allows more violations and imposes stronger regularization.
- Choose C using validation data.

Examples inside the margin, including misclassified examples, can influence the fitted boundary.

The SVM decision score can rank documents, but it is not a probability.

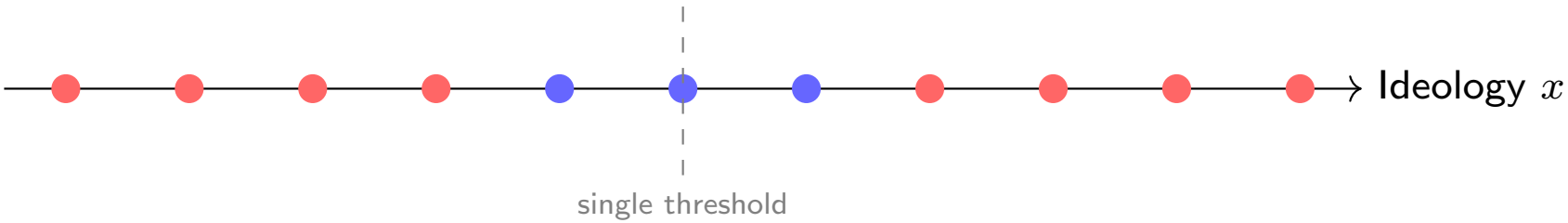
When Linear Boundaries Fail

- Linear classifiers (like SVMs) can't separate all datasets.
- Example: circular data: no straight line can split inner and outer classes.
- Need to think beyond linear boundaries.



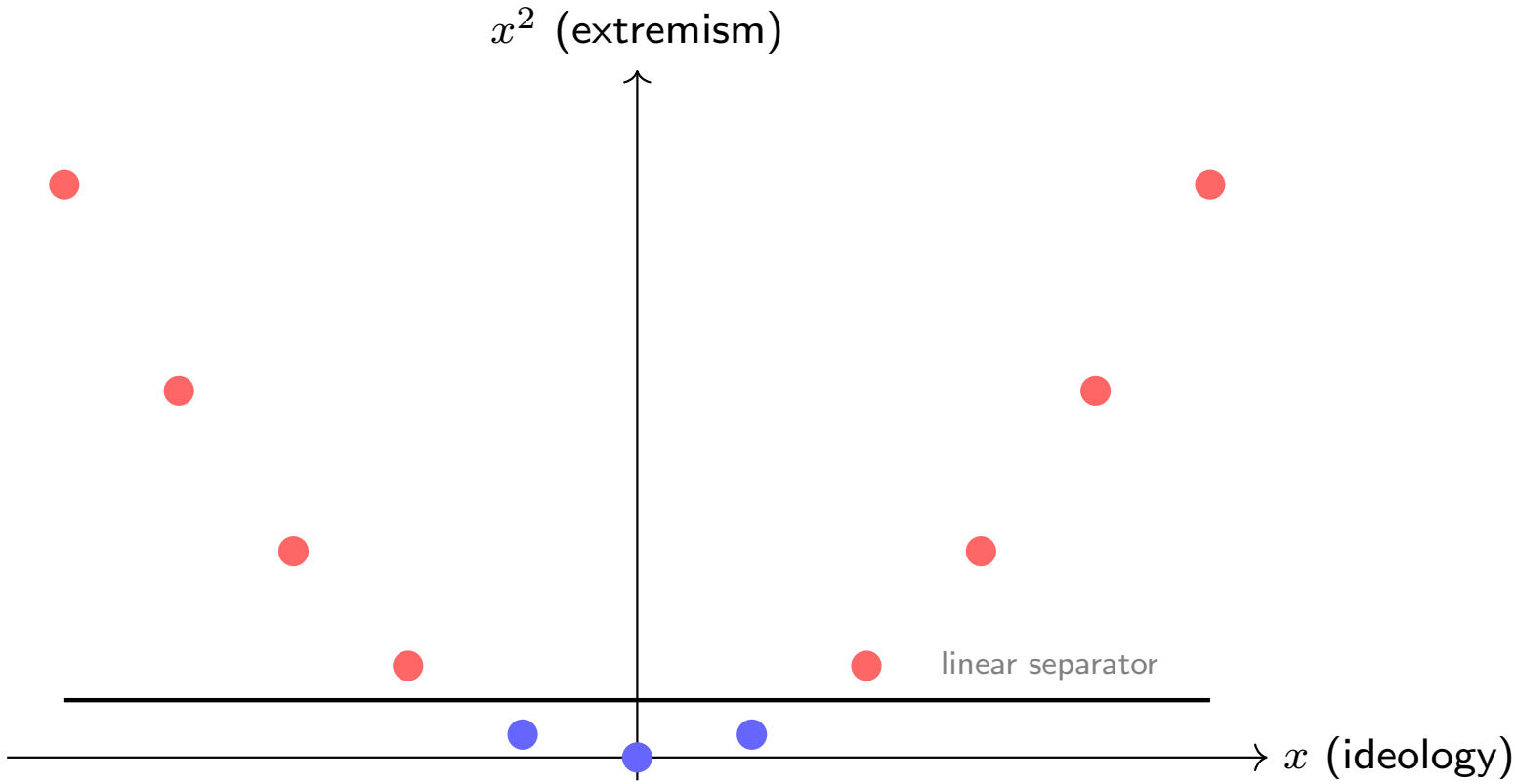
No linear hyperplane in 2D can separate these points.

Ideology in 1D: Not Linearly Separable



Extremes (red) lie on both sides of the center — one cut on a line can't separate them from moderates (blue).

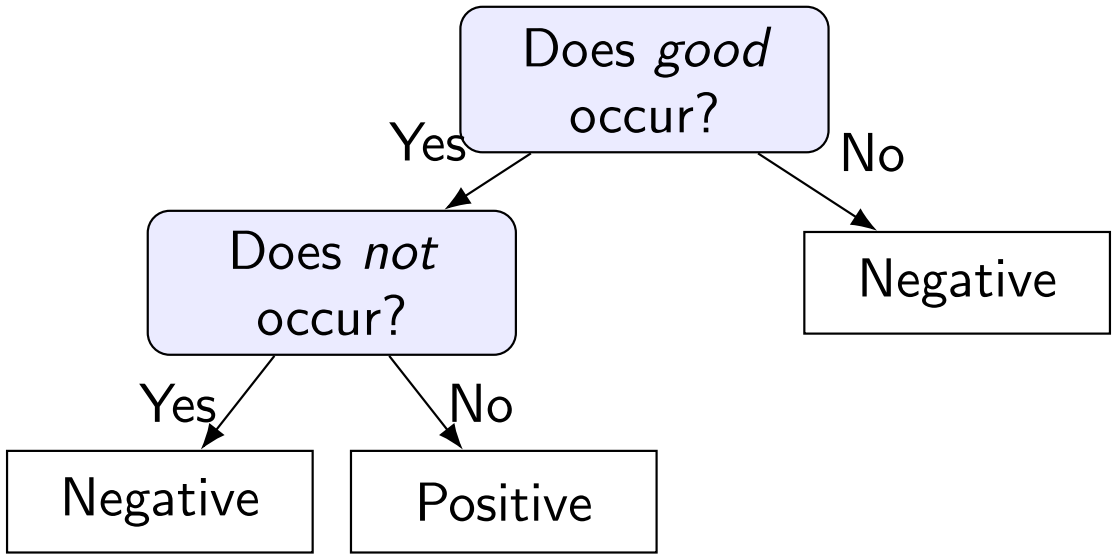
Nonlinear Map $\phi(x) = (x, x^2)$: Now Linearly Separable



After $\phi(x)$, a horizontal linear boundary separates extremes (red) from moderates (blue).

Decision trees

A tree predicts through a sequence of feature-based questions. The splits are learned from labelled examples.



Illustrative tree. A classification leaf predicts a class; a regression leaf predicts a numerical value.

Controlling a tree's complexity

A deep tree can fit small details in the training sample, including rare words or names that will not generalize.

Two useful controls are:

- limiting the tree's depth
- requiring enough training examples in each leaf

Choose these settings using validation data. Read the rules to see which features the tree relies on.

Combining trees into an ensemble

An ensemble combines several fitted models instead of relying on one tree.

Illustrative tree	Predicted class
Tree 1	Positive
Tree 2	Positive
Tree 3	Negative
Majority vote	Positive

For classification, combine votes or probabilities. For regression, average numerical predictions.

Bagging and random forests

- **Bagging:** fit trees on different bootstrap samples of the training data, then combine their predictions.
- **Random forests:** also consider a random subset of features at each split.

Making trees less similar can reduce the variation of their combined prediction compared with a single tree.

The combined model is harder to explain as one rule, so inspect its performance and the features it relies on.